5

159.302 CSP

Solving CSPs

Source of contents: ***MIT OpenCourseWare***

Solving CSPs

5

Approaches to solving CSPs are some combination of **constraint propagation** and **search**.

1. **Constraint propagation –** to eliminate values that could not be part of any solution

2. **Search –** to explore valid assignments

Solving CSPs

Constraint Propagation (*aka* Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i, V_j) is arc **consistent** if

$\forall\, x \in D_i \; \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

For **every**

there exists **some**

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i , V_j) is arc consistent if

$\forall\, x \in D_i\ \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from D_i (domain of variable at tail of constraint arc) that fail this condition.

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$$V_i \rightarrow V_j$$

Directed arc (V_i, V_j) is arc consistent if

$\forall\, x \in D_i\ \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from D_i (domain of variable at tail of constraint arc) that fail this condition.

Assume domains are of size **d** at the most, and there are **e** binary constraints.

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i, V_j) is arc consistent if

$\forall\, x \in D_i\ \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from $\mathbf{D_i}$ (domain of variable at tail of constraint arc) that fail this condition.

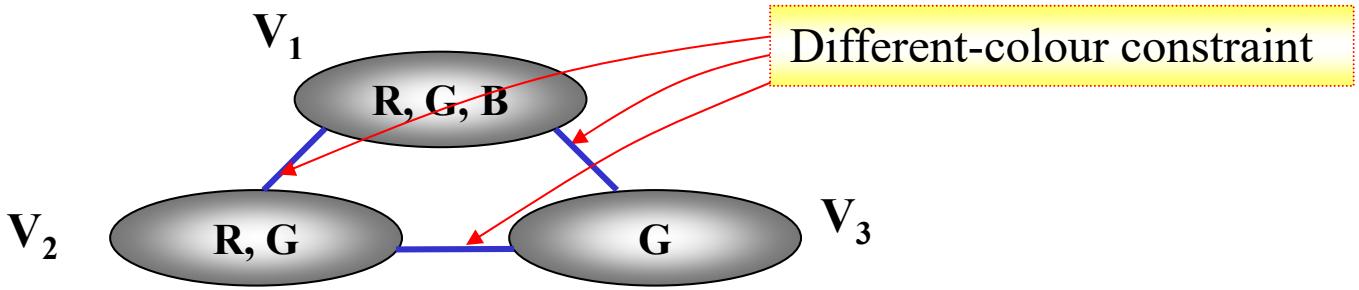
Assume domains are size at most **d** and there are **e** binary constraints.

In the **worst case**, a simple algorithm for arc consistency requires $\mathbf{O(ed^3)}$ arc tests– note that just verifying arc consistency takes $\mathbf{O(d^2)}$ for each arc.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

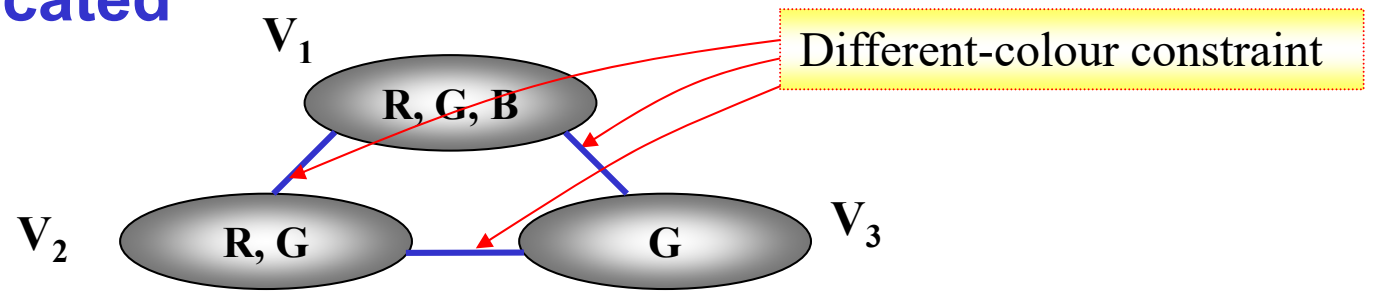


- Each variable is constrained to have values different from its neighbors

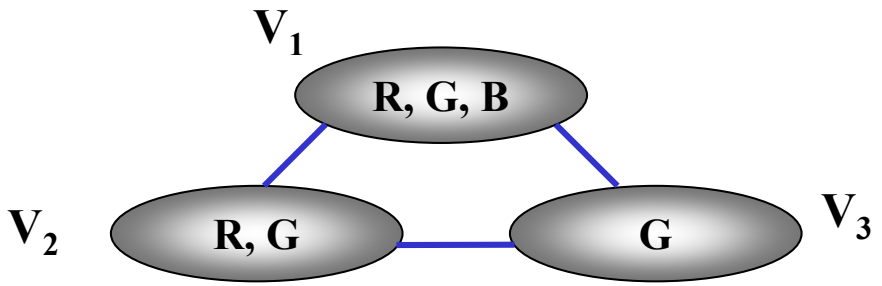
Constraint Propagation Example

Graph Colouring

Initial domains are indicated



Arc examined	Value deleted



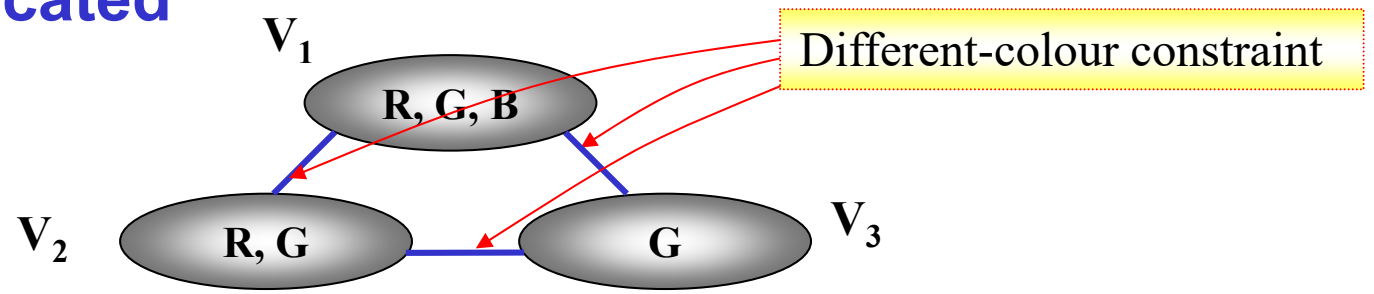
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

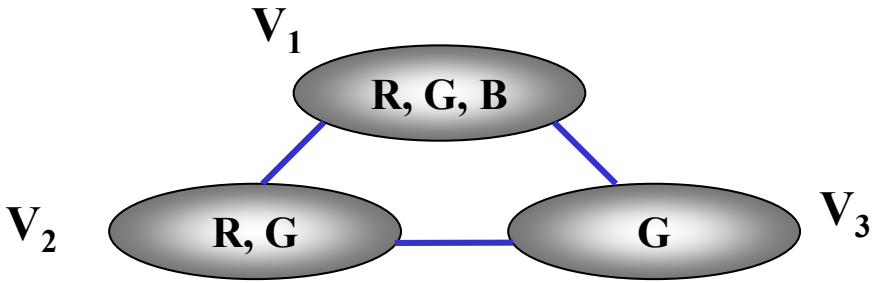
Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none





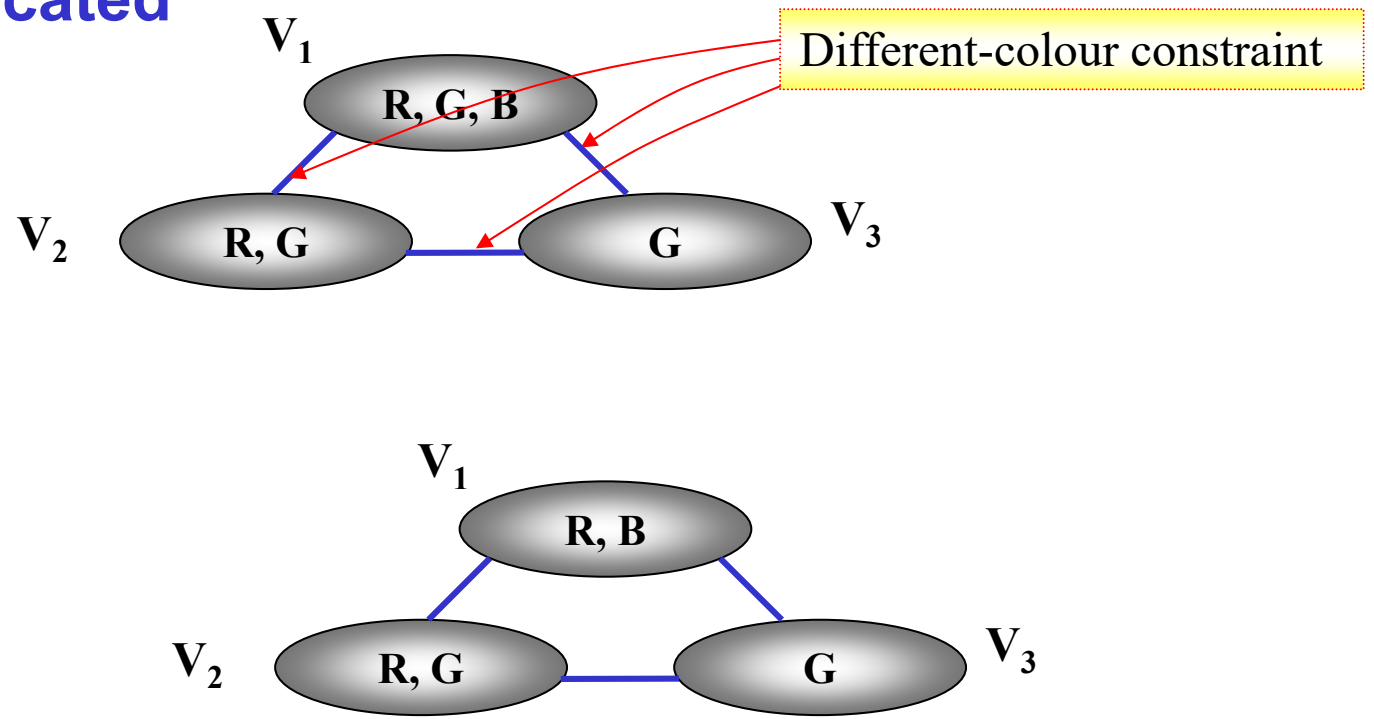
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)



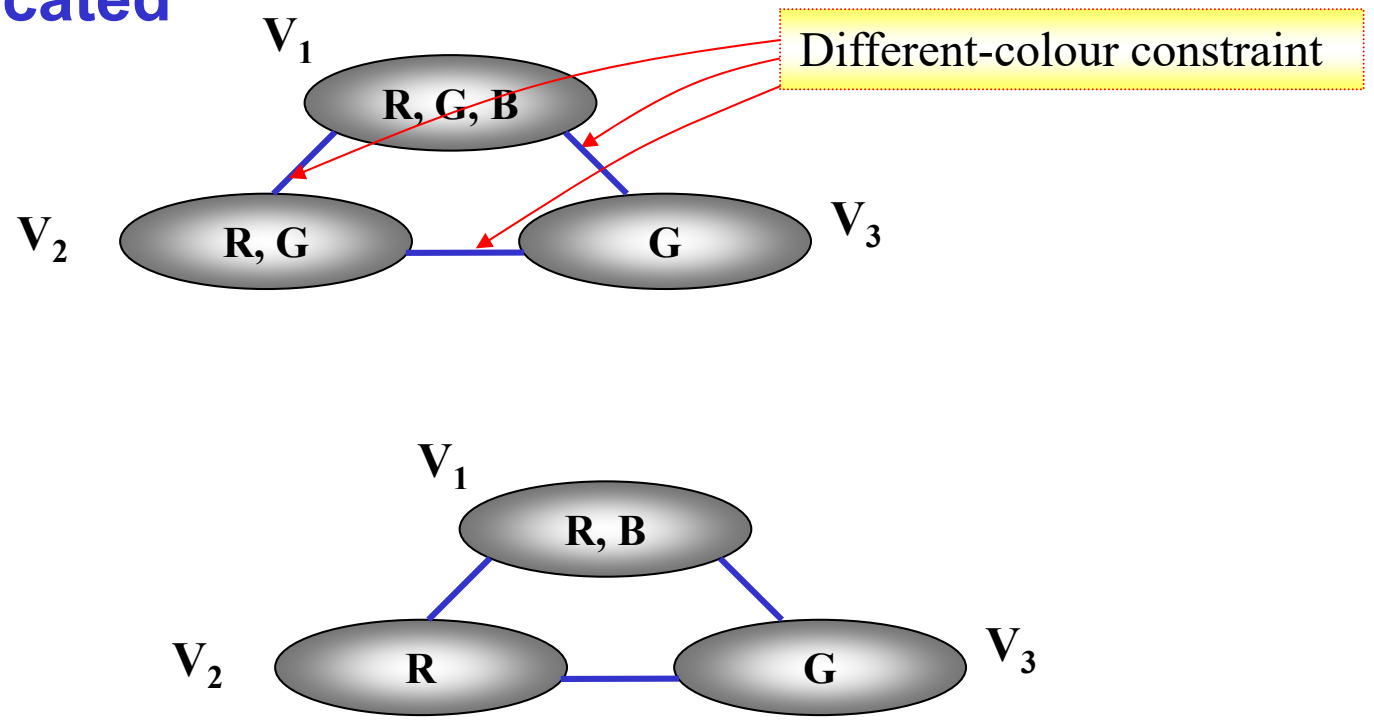
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)
V_2-V_3	V_2(G)



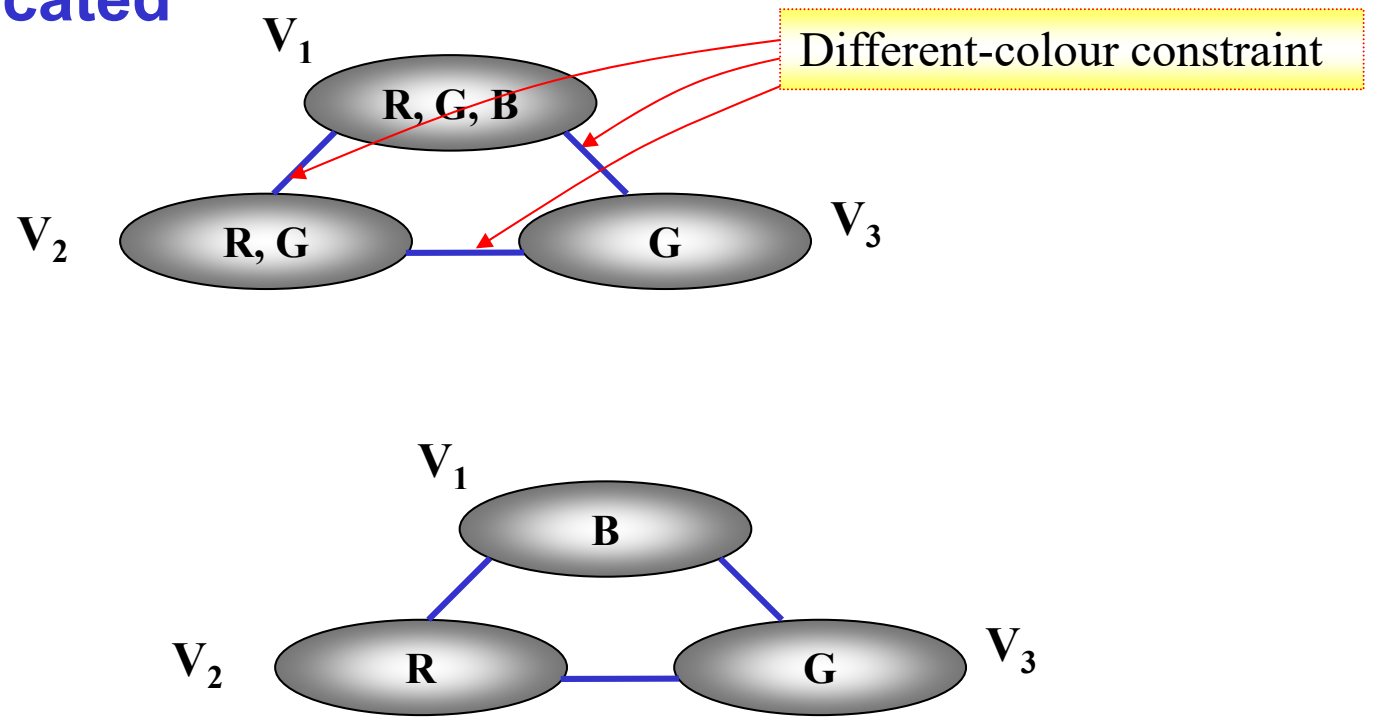
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

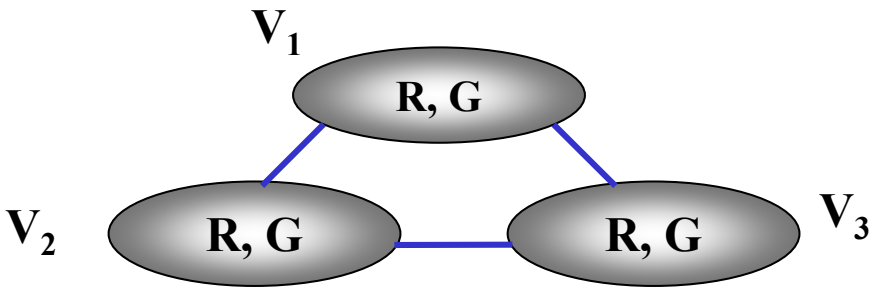
Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)
V_2-V_3	V_2(G)
V_1-V_2	V_1(R)
V_1-V_3	none
V_2-V_3	none



- In general we need to make one pass through any arc whose head variable has changed until no further changes are observed before we can stop.

But, arc consistency is not enough in general!

Graph Colouring

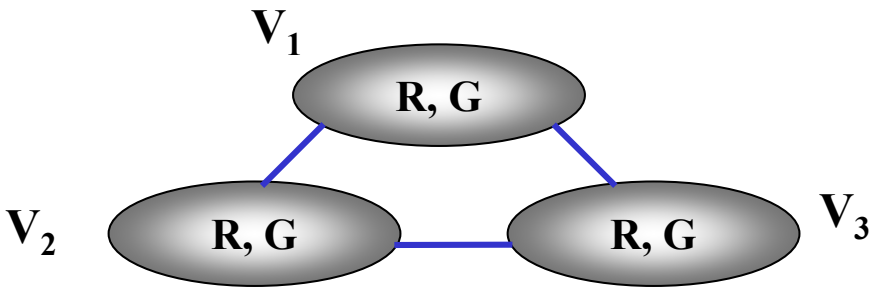


- Arc consistent but **NO SOLUTION**

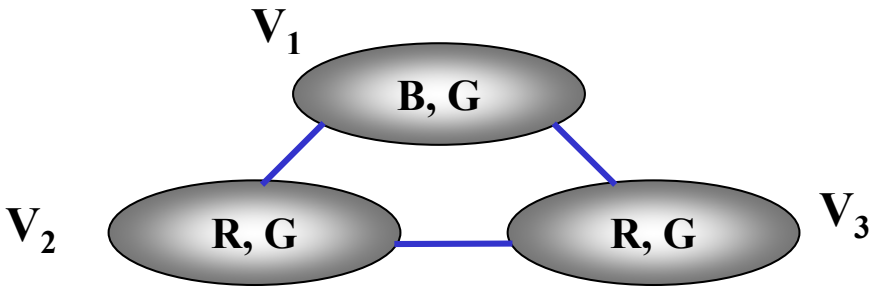
We need one colour for each variable!

But, arc consistency is not enough in general!

Graph Colouring



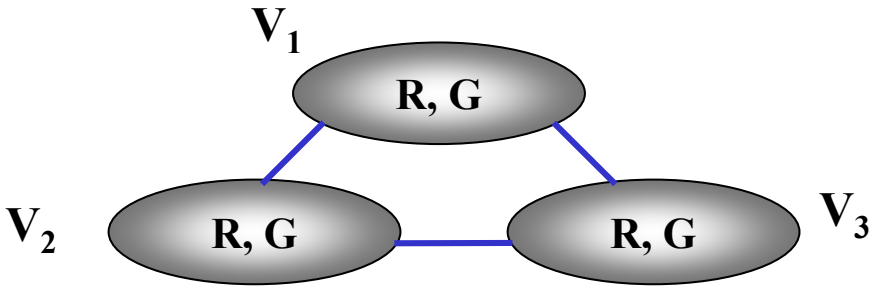
- Arc consistent but **NO SOLUTION**



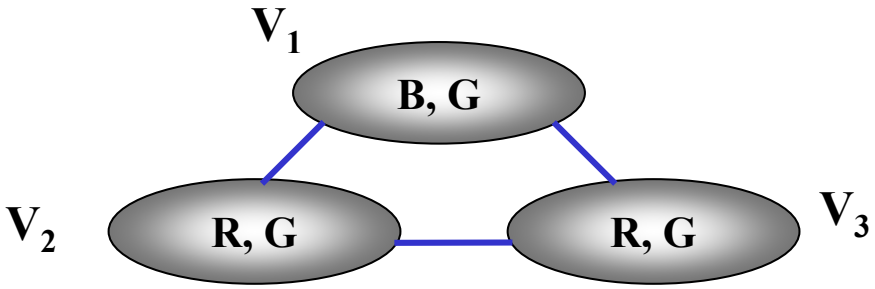
- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R

But, arc consistency is not enough in general!

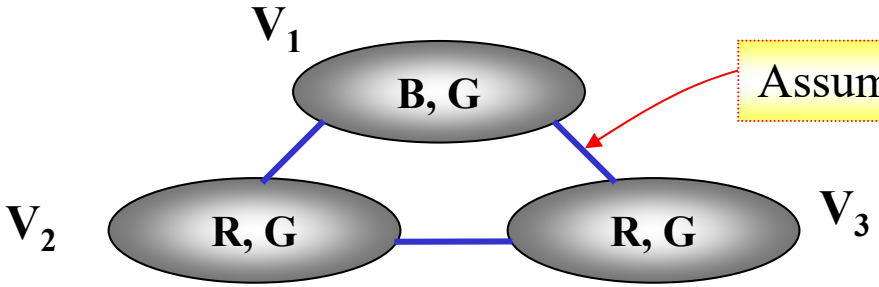
Graph Colouring



- Arc consistent but **NO SOLUTION**



- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R



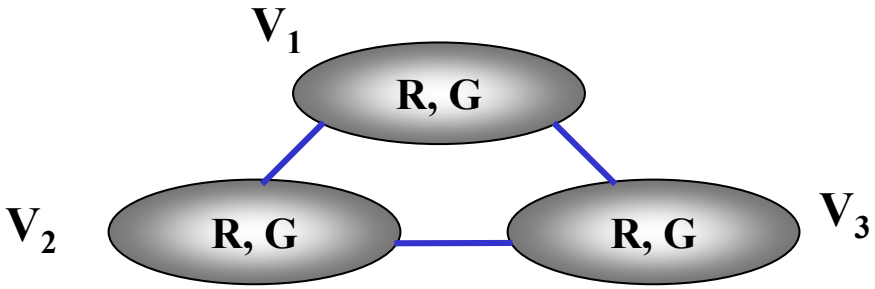
Assume B, R not allowed

- Arc consistent, **1 SOLUTION by virtue of a special constraint**

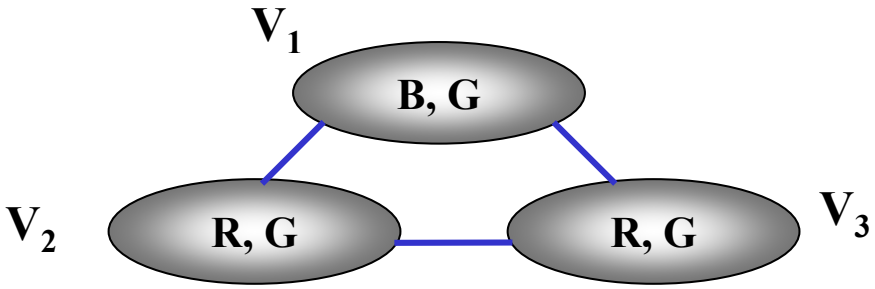
CSP

But, arc consistency is not enough in general!

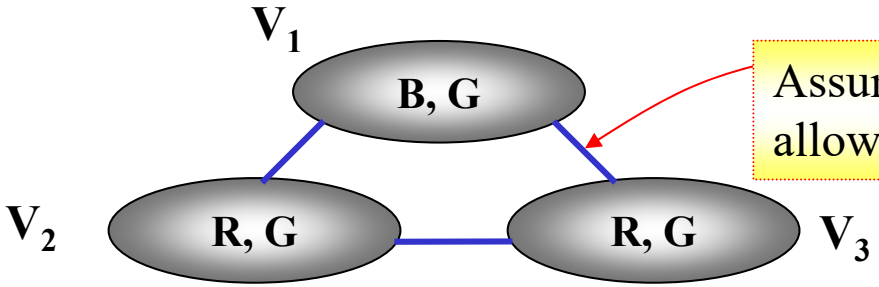
Graph Colouring



- Arc consistent but **NO SOLUTION**

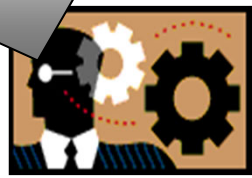


- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R



Assume B, R not allowed

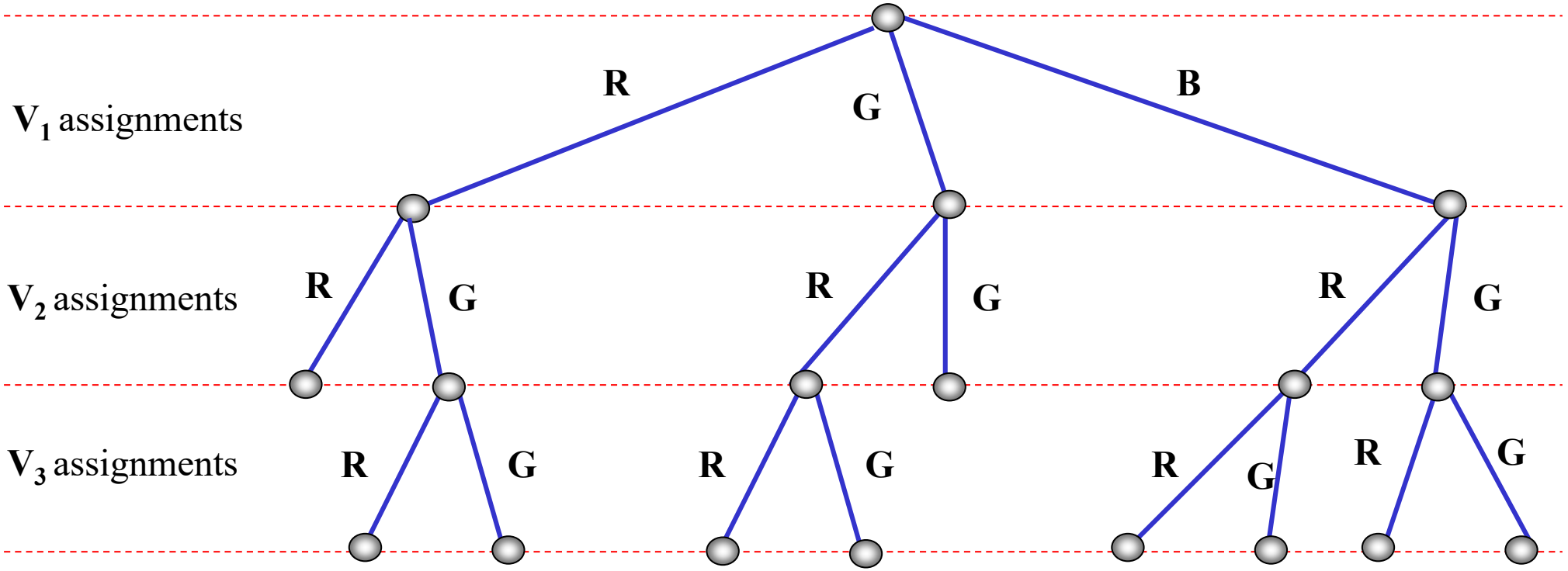
In general, if there is more than one value in the domain of any of the variables, we need to apply **Search** algorithms to find solutions (if there is any)



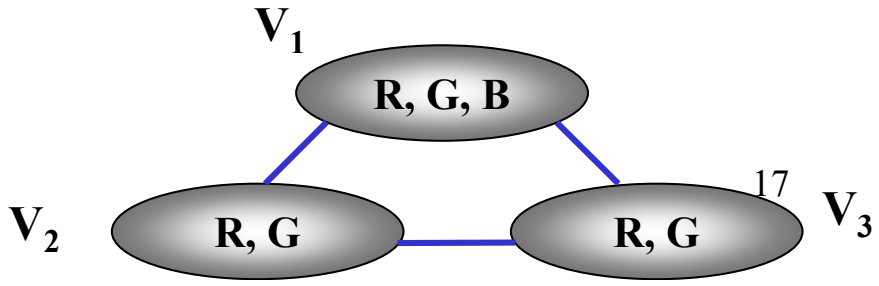
- Arc consistent, **1 SOLUTION**

Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).

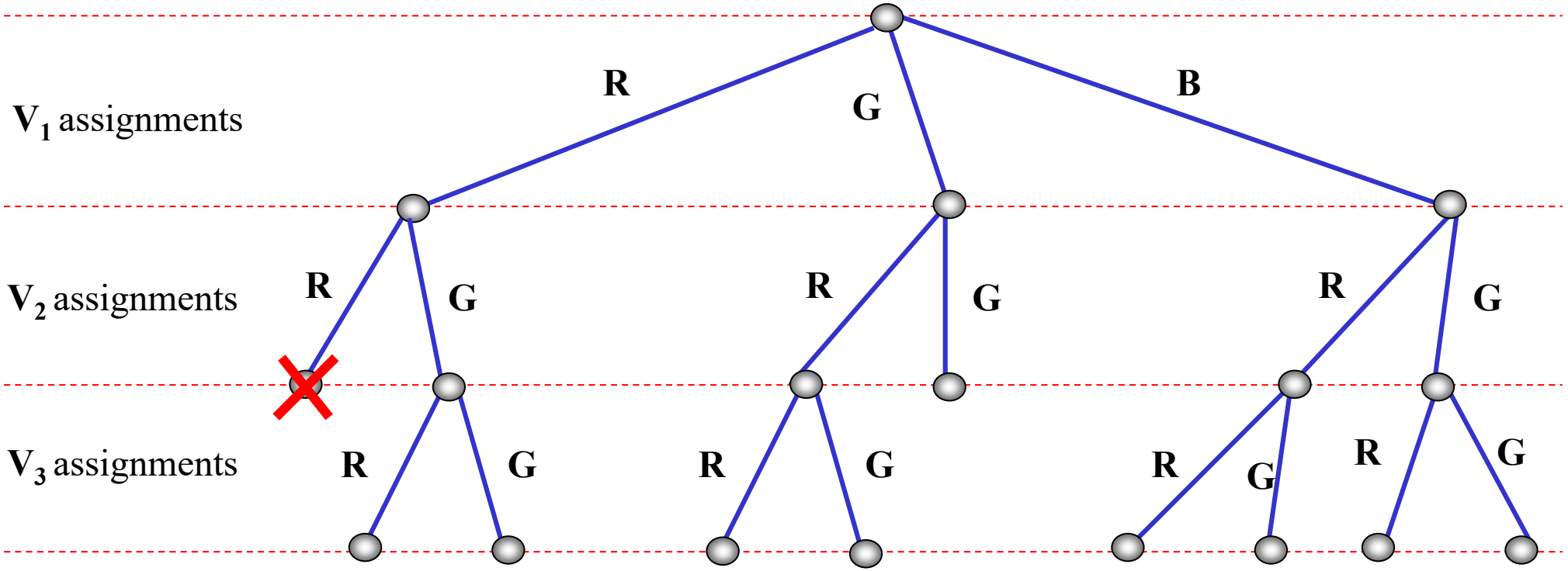


Note: Do not alphabetise the enumeration of values.



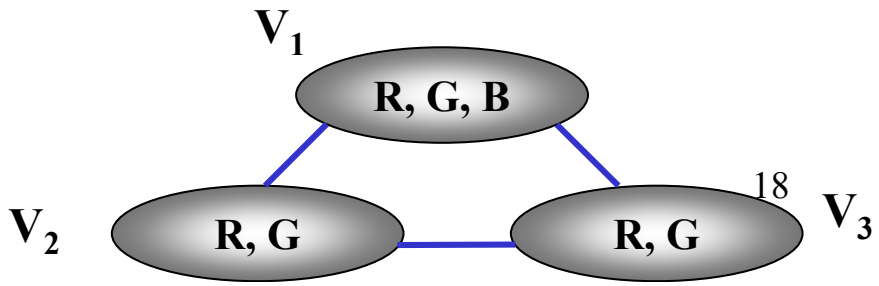
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



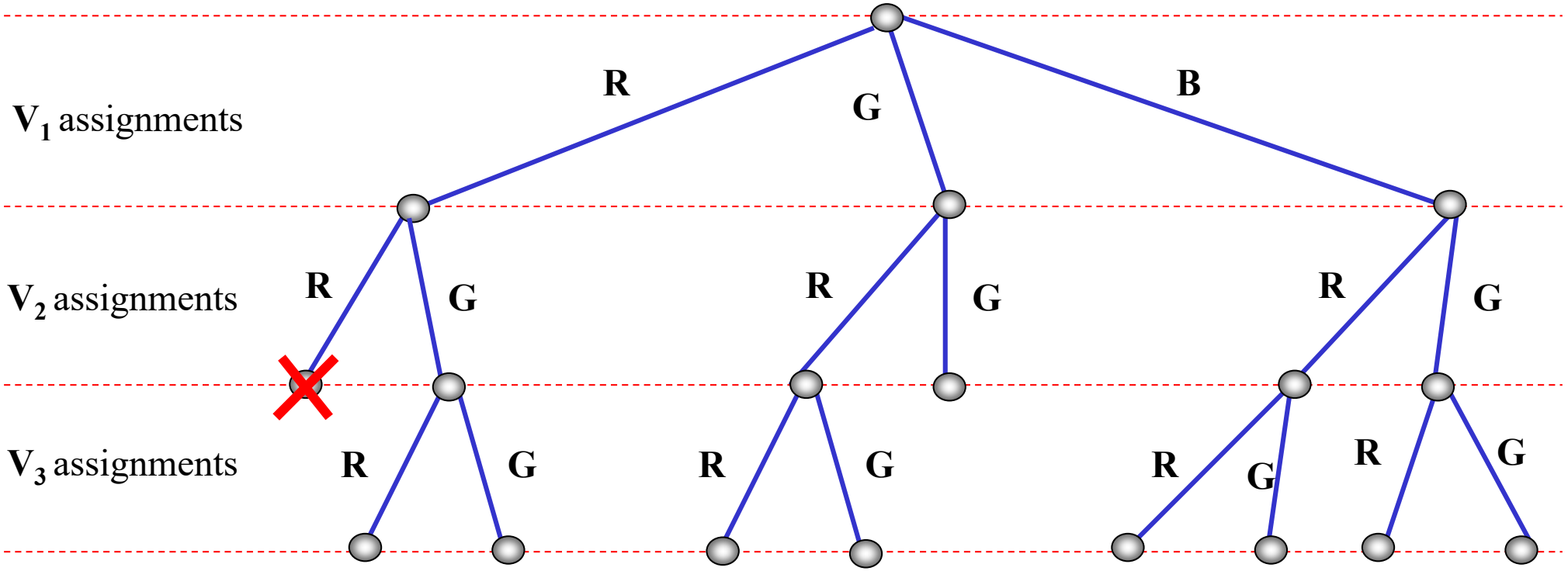
Inconsistent with V_1 = **R**

Backup at inconsistent assignment.



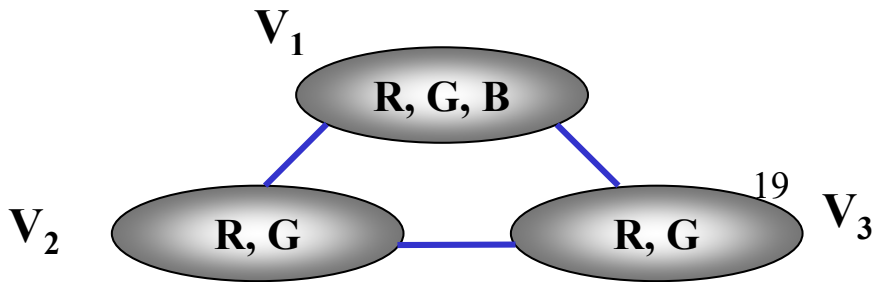
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



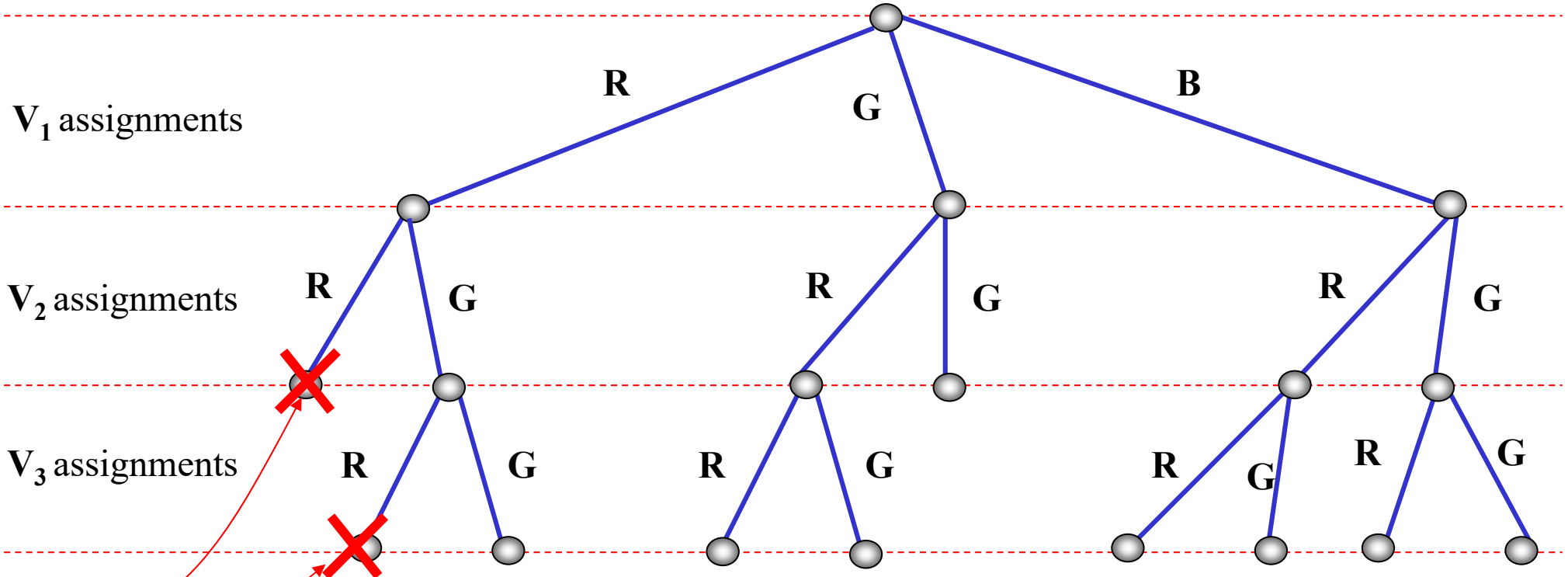
Inconsistent with V_1 = **R**

Backup at inconsistent assignment.



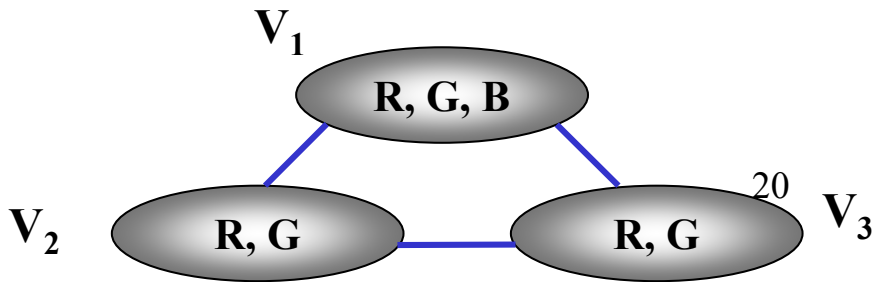
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



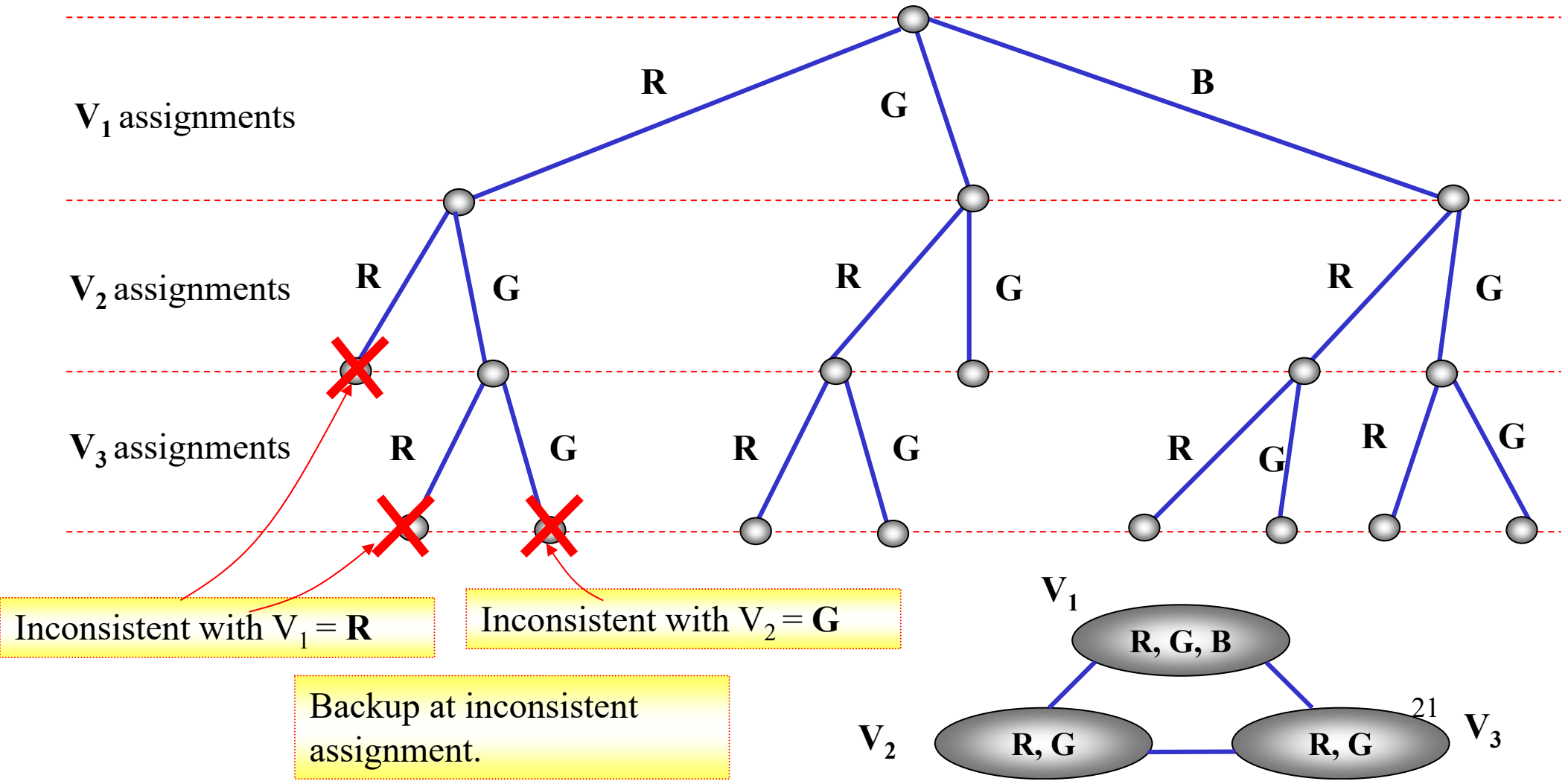
Inconsistent with V_1 = **R**

Backup at inconsistent assignment.



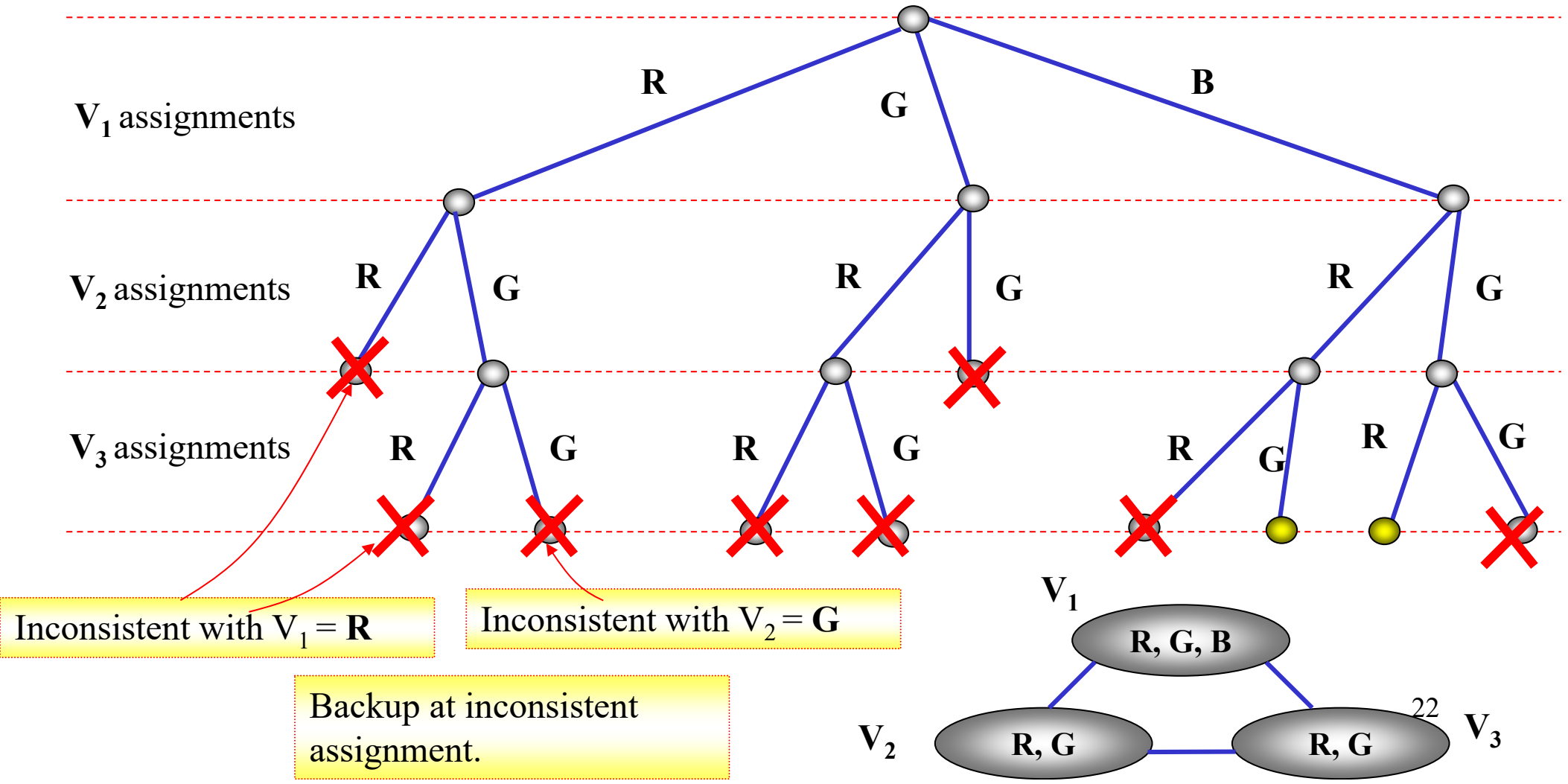
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



Solving CSPs

Combine Backtracking & Constraint Propagation

How can we improve Pure BT when searching for a solution?

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

Solving CSPs

Combine Backtracking & Constraint Propagation

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

How much propagation to do?



Solving CSPs

Combine Backtracking & Constraint Propagation

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

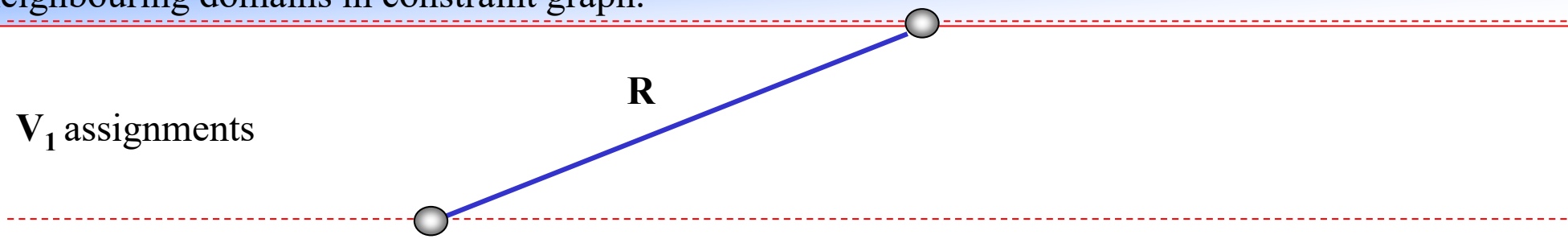
Answer: Not much, just local propagation from domains with unique assignments, which is called **forward checking (FC)**. This conclusion is not necessarily obvious, but generally holds in practice.

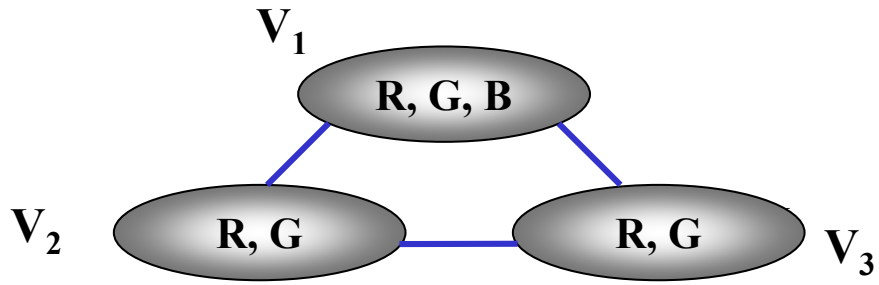
How much propagation to do?



Backtracking with Forward Checking (BT-FC)

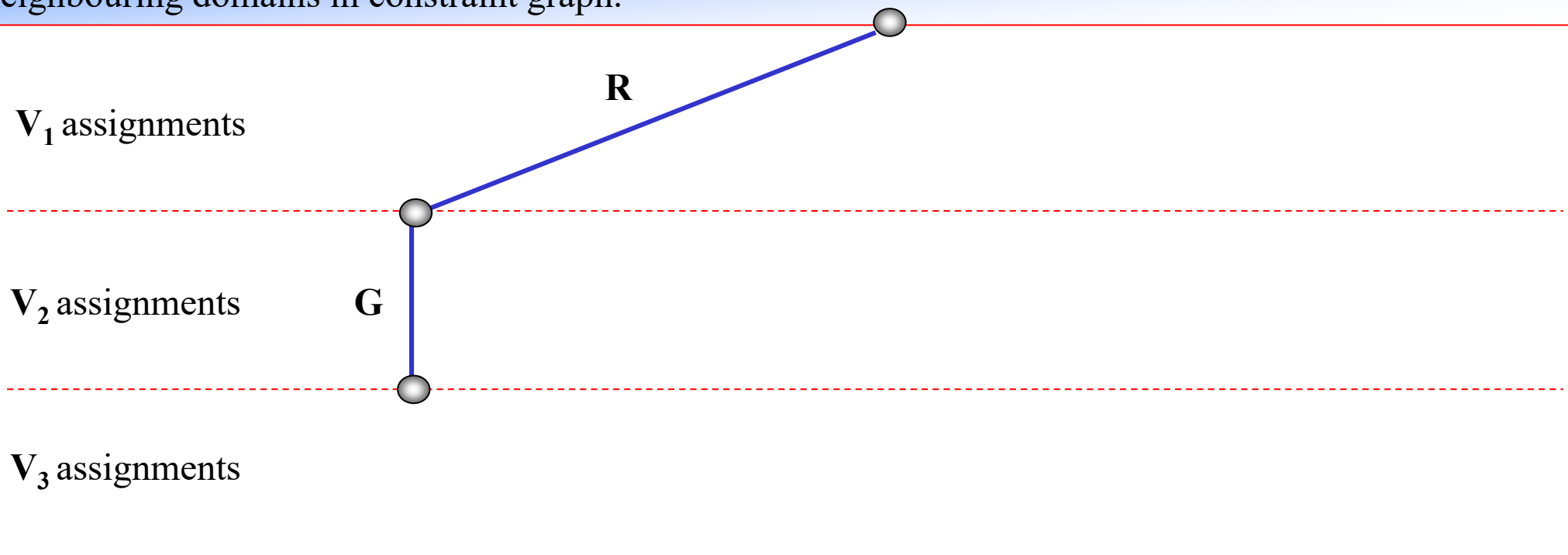
When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.



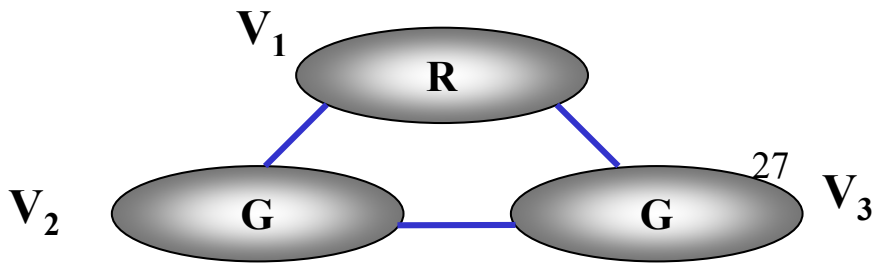


Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

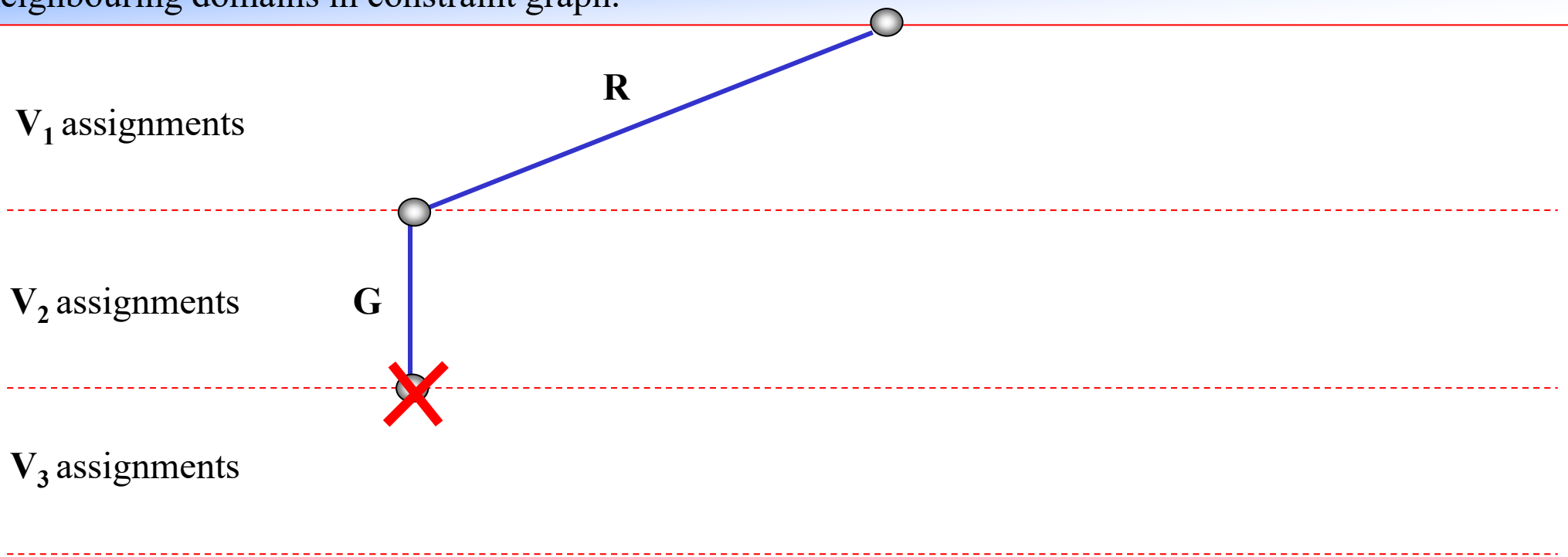


We eliminate any values that are inconsistent with the assignment.

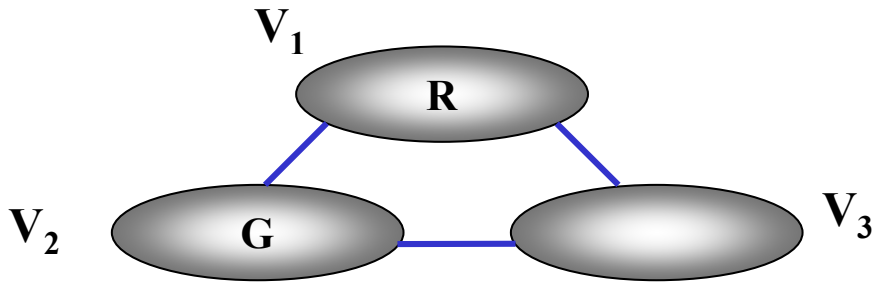


Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

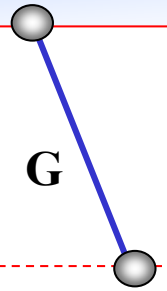


We have a conflict whenever a domain becomes empty.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

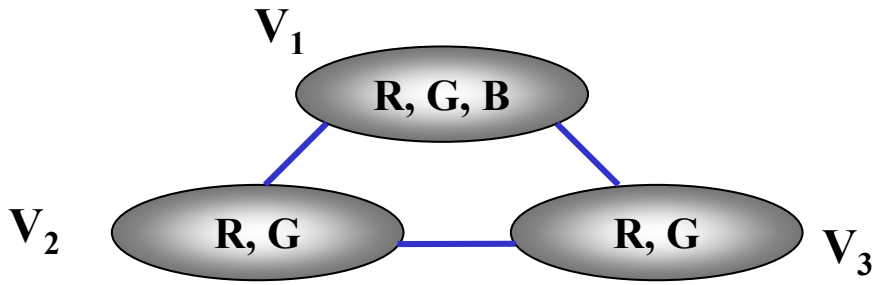


V_1 assignments

V_2 assignments

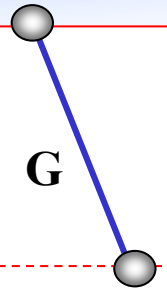
V_3 assignments

When backing up, we need to restore domain values, since deletions were done to reach consistency with tentative assignments considered during search.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

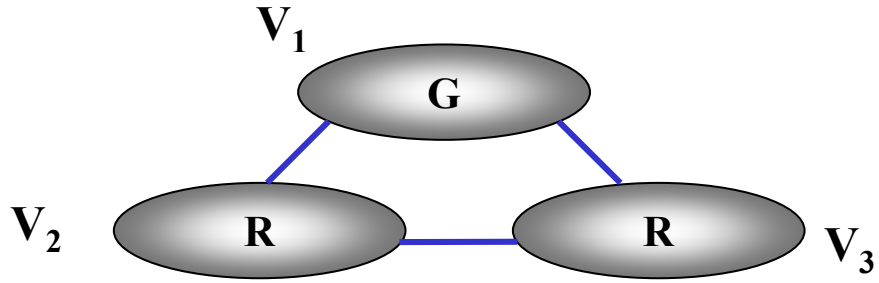


V_1 assignments

V_2 assignments

V_3 assignments

We eliminate G from V_2 and V_3.



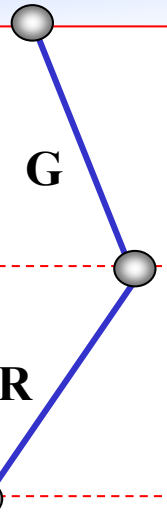
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

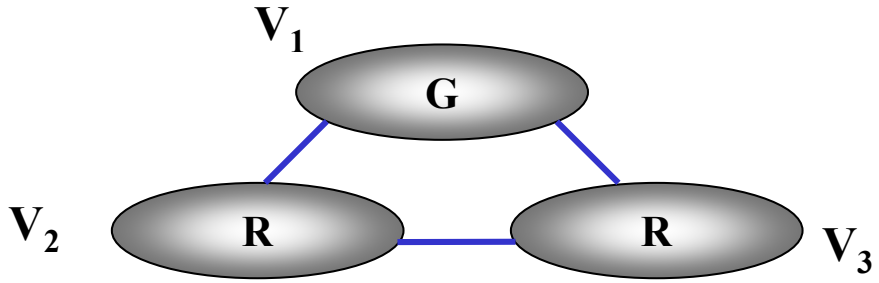
V_1 assignments

V_2 assignments

V_3 assignments



We now consider V_2 = R and propagate.



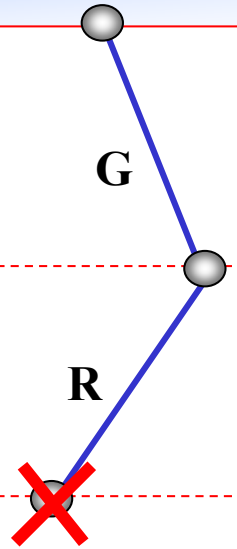
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

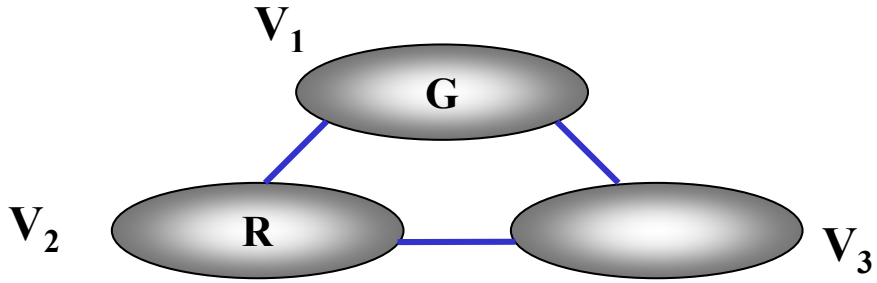
V_1 assignments

V_2 assignments

V_3 assignments

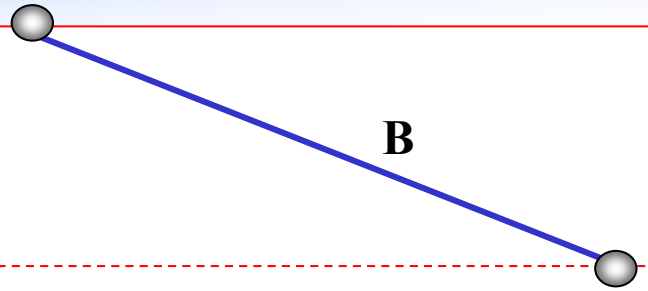


The domain of V_3 is now empty and so we fail and backup.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

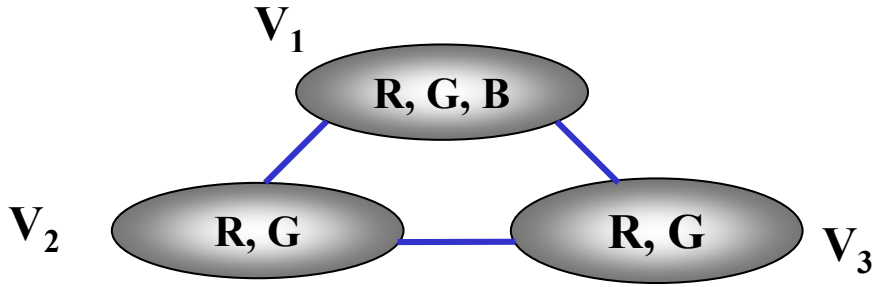


V_1 assignments

V_2 assignments

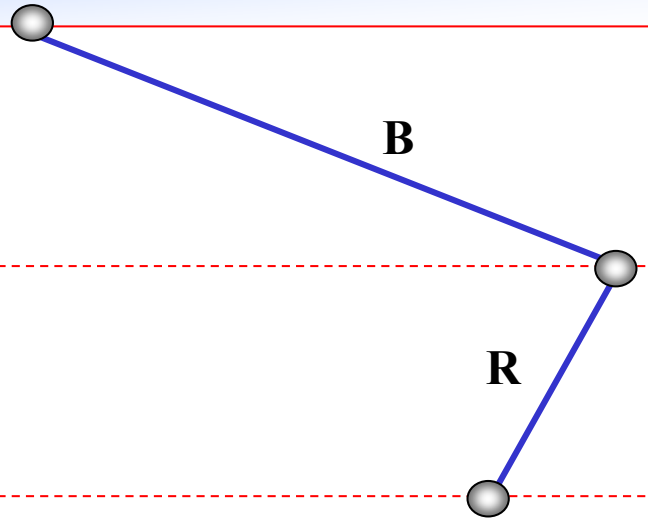
V_3 assignments

So, we move to consider V_1 = B and propagate.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

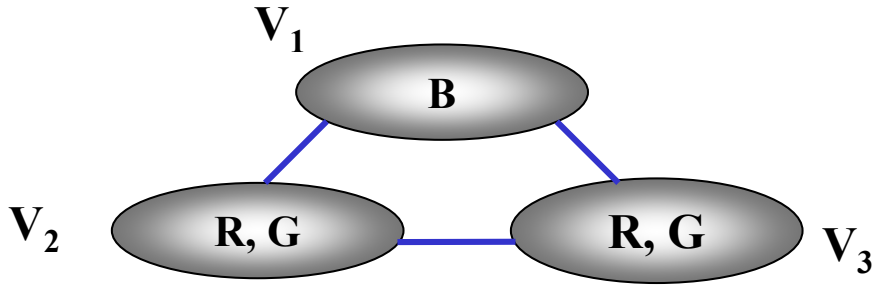


V_1 assignments

V_2 assignments

V_3 assignments

The propagation does not delete any values. We pick V_2 = **R** and propagate.



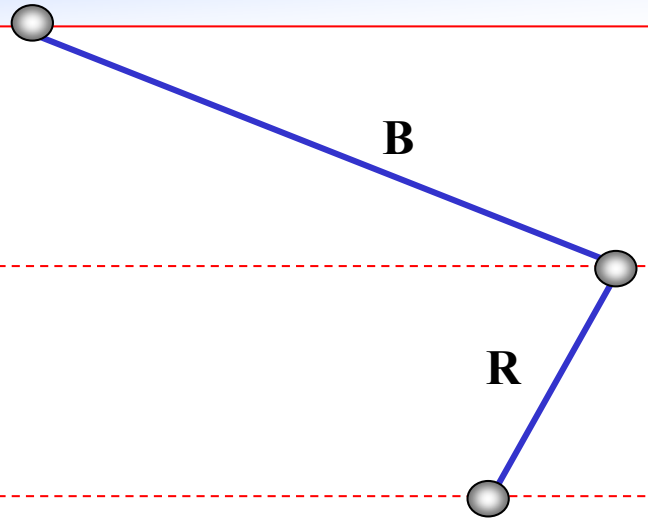
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

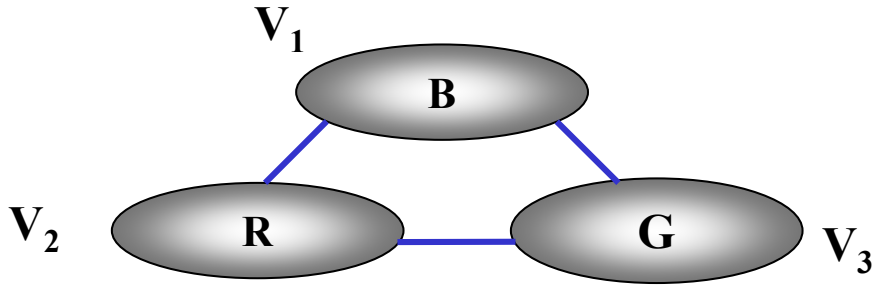
V_1 assignments

V_2 assignments

V_3 assignments



This removes the R values in the domains of V_1 and V_3.



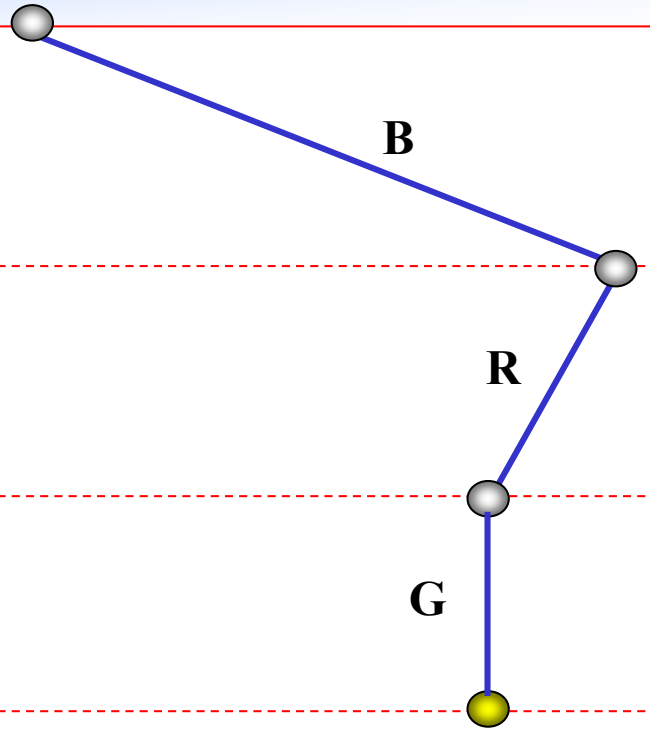
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

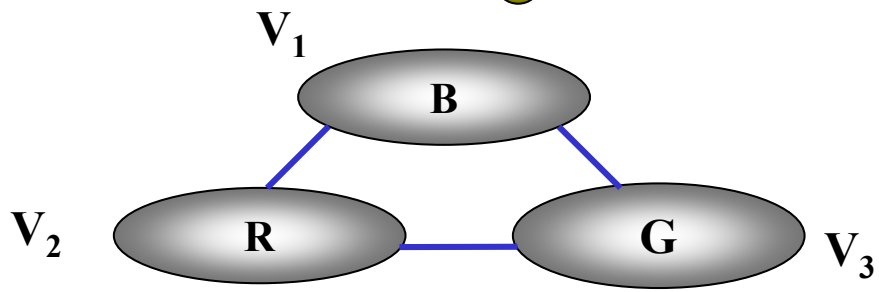
V_1 assignments

V_2 assignments

V_3 assignments



We pick V_3 = G and have a consistent assignment.



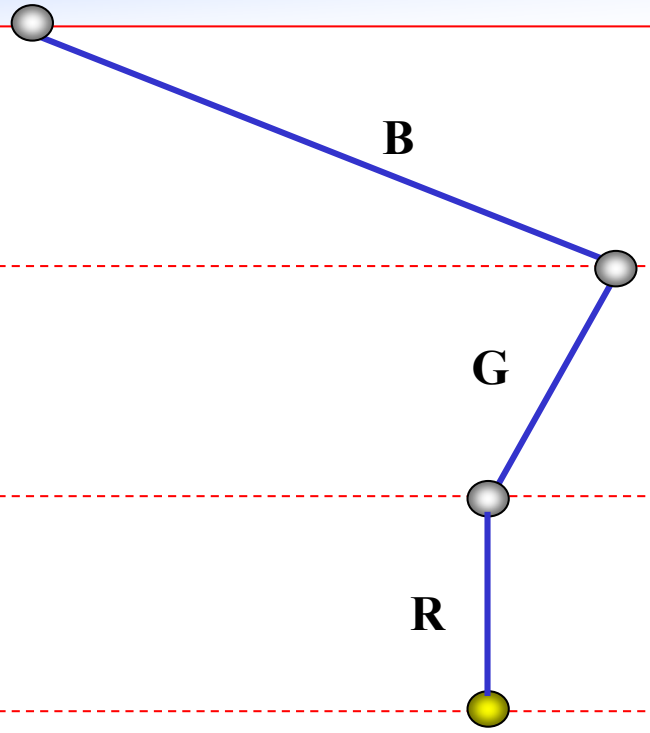
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

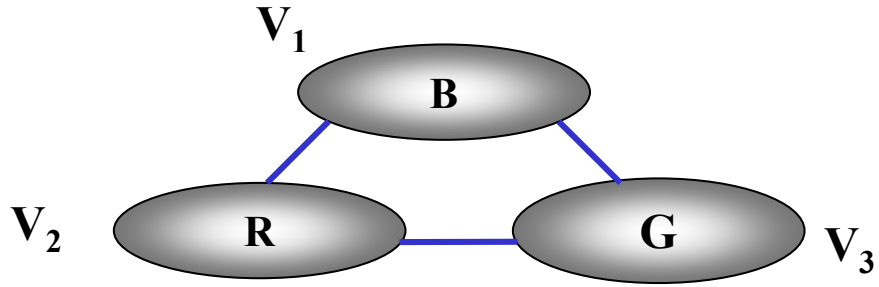
V_1 assignments

V_2 assignments

V_3 assignments

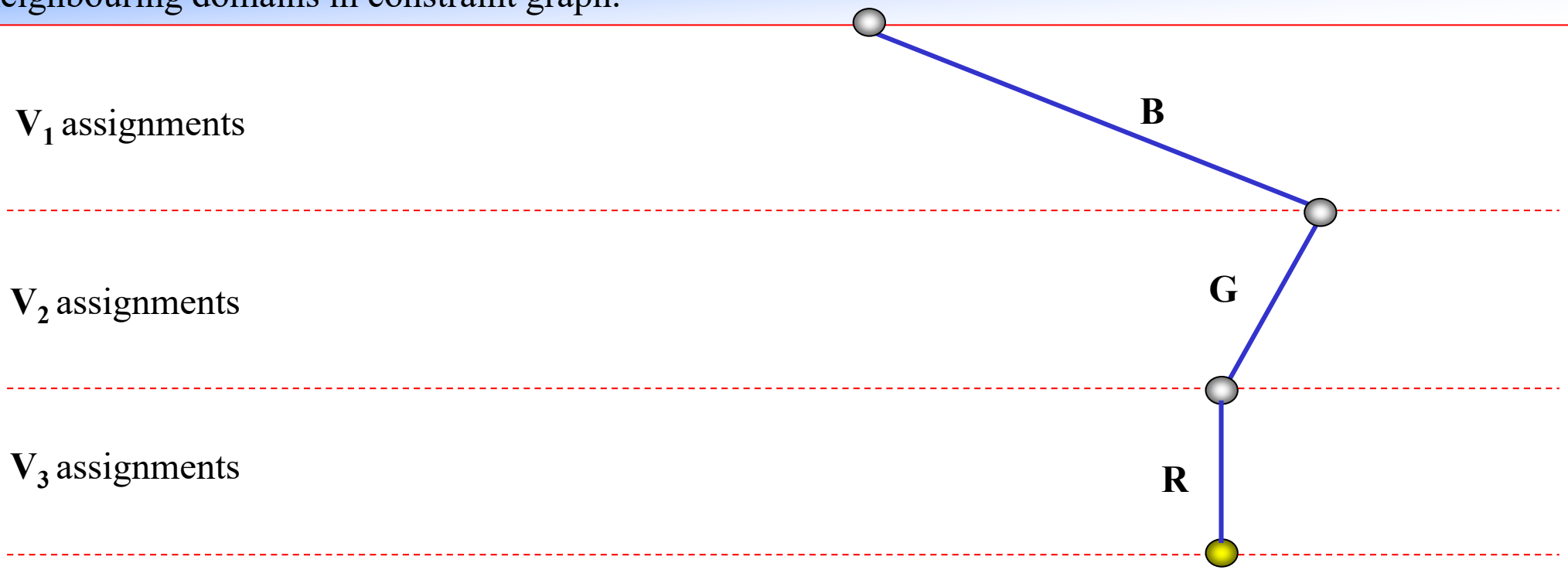


We can continue the process to find the other consistent solution.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.



No need to check previous assignments

Generally preferable to pure BT.

